weights	c_{trunc}	c_{term}	$c_{c^\perp}$	$c_{c^\parallel}$	c_v	c_d	c_f
Value (sim.)	-1	0.1	0.1	0.05	0.3	0.15	0.4
Value (real)	-1	0.03	0.1	0.05	0.35	0.2	0.3

Table A.1. Reward weightings for the simulated and real hardware environment.

reward	$r_{c^\perp}$	$r_{c^\parallel}$	r_v	r_d	r_f	r_f
boundaries	e_{max} [m]	e_{max} [m]	e_{max} [$\mathrm{m\,s^{-1}}$]	e_{max} [rad]	e_{max} [N]	e_{mid} [N]
Value (sim.)	0.01	0.01	0.01	0.6	2.0	0.5
Value (real)	0.015	0.03	0.03	0.6	4.0	1.5

Table A.2. Reward transformation bounds for the simulated and real hardware environment.

Environment	Simulation		Real Hardware
Parameter	Value (SAC)	Value (SAC-Ensemble)	
optimizer	ADAM	ADAM	ADAM
η	$1 \cdot 10^{-4}$	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
γ	0.99	0.99	0.99
ξ	0.005	0.005	0.005
$\lvert\mathcal{D}\rvert$	10^6	10^6	10^6
hidden-layers (critic NNs)	$[256, 256]$	$[256, 256]$	$[256, 256]$
hidden-layers (policy NN)	$[256, 256]$	$[256, 256]$	$[256, 256]$
# of critic networks	2	5	10
batch-size	256	256	256
UTD	-	1	2
non-linearity	ReLU	ReLU	ReLU
α	0.2	0.2	0.2
α-tuning	True	True	True
initial random steps	15000	15000	15000

Table A.3. Hyperparameters for the SAC agents.

Parameter	Value (sim.)	Value (real)
$\lambda_{\min}$	0.2	0.2
$\lambda_{\max}$	1.0	1.0
$u_{\min}$	0.02	0.015
$u_{\max}$	0.2	0.1
horizon T	380	150

Table A.4. Hyperparameters of CHEQ used for the weight adaption function λ^{RL} for the hybrid agent.

Environment	Simulation		Real Hardware
Parameter	baseline	suboptimal	
a_r [m]	0.016	0.2	0.015
$\Delta x^{\mathrm{indent}}$ [m]	0.012	0.015	0.002
spacing [m]	$5 \cdot 10^{-4}$	$5 \cdot 10^{-4}$	$5 \cdot 10^{-4}$

Table A.5. Hyperparameter choices for the nominal controller for the simulation and the real hardware environment. In addition, we define the suboptimal controller that is used for CHEQ. As optimal gains for the real process are unknown, we label the parameters for the real process as suboptimal as well.

control points as close control points P^{close}. Further, note that we use a low value for the indentation $\Delta x^{\mathrm{indent}}$ in the real environment. This is due to the fact that the movement of the real robot is more cumbersome than in the simulated environment. Setting a lower indentation leads to a stronger movement in the direction of the path.

Baseline and Impedance Gains. To define our baseline control (*Nominal (tuned)*), we perform hyperparameter tuning with Bayesian Optimization to find the optimal static impedance gains for the baseline controller in simulation. As we expect the important impedance gains to be k_y and k_z, we use these values and the damping factor ζ for the search space. We keep the other impedance gains fixed at values that are high enough to perform the given movement by the nominal controller. We choose $k_x = 500\,\mathrm{N\,m^{-1}}, k_{\phi_x} = k_{\phi_y} = k_{\phi_z} = 500\,\mathrm{N\,rad^{-1}}$. For a better comparison of the learning capabilities, we define the same bounds for the search space as provided by the action space of the RL agent. Thus the search space is spanned as $k_y \in [50, 200]\,[\mathrm{N\,m^{-1}}]$, $k_z \in [30, 130]\,[\mathrm{N\,m^{-1}}]$, and $\zeta \in [0.8, 1.2]$. We perform a total of 2000 runs and rank them according to the predefined score of the task

$$\text{score} = w_c \cdot \frac{N_{\mathrm{con}}}{N_{\mathrm{tot}}} + w_w \cdot \frac{n_{\mathrm{wipes}}}{7} - w_f \cdot \overline{\Delta F_N} - w_v \cdot \overline{\Delta v_T},$$

where N_{tot}, N_{con} are the numbers of total episodic steps and steps with contact to the workpiece, n_{wipes} describes the number of wiped wiping points, and $\overline{\Delta F_N}$, $\overline{\Delta v_T}$ are the average deviations of force and velocity to the specified targets. The constant values at the beginning of each score term are the weighting factors. Since we favour runs

that polish over the complete task space in the predefined horizon T, we set a relatively high weighting to the wiping and contact term. The weighting factors are configured as $w_c = 0.18$, $w_w = 0.52$, $w_f = 0.03$, and $w_v = 0.27$. In the end, we choose the run resulting in the highest score as the impedance gains of the *Nominal (tuned)* controller, that could be found after ~ 500 runs. For the control prior used for the CHEQ agent, we further define a suboptimal controller (*Nominal (untuned)*) determined by rounding up the values.

For the partially tuned nominal controller *Nominal (tuned section-wise)*, we divide the workpiece into 5 distinct sections. The division of the task is based on the current y-position of the end-effector. We find the gains for the partitioned controller by iteratively tuning with Bayesian Optimization for each section and using the best-found gains for the previous sections. Table A.7 provides the final impedance gains configurations using this iterative method.

Appendix B. ADDITIONAL SIMULATION RESULTS

This section describes additional results.

B.1 Comparison of Tuned Baselines with Trained CHEQ Agent

For a better comparison, we show the results of our trained CHEQ agent in comparison with the three nominal control baselines *Nominal (untuned)*, *Nominal (tuned)* and *Nominal (tuned section-wise)* in Fig. B.1. In the first row, we compare the untuned controller, the tuned fixed gain controller, the section-wised tuned controller and our trained CHEQ agent. The second row shows the same comparison but leaves out the untuned controller to enable a better comparison. We compare the force and velocity behavior over one full episode in the subfigures B.1a and B.1d. The force and velocity boxplots for the five workpiece sections are shown in the subfigures B.1b, B.1c, B.1e, and B.1f.

Appendix C. ADDITIONAL REAL-WORLD RESULTS

This section describes additional experimental details and results for the hardware experiments.

Real-world RL Challenges and Solutions. Additional challenges need to be considered to achieve stable training on hardware. We found (i) control frequency, (ii) sensor noise and (iii) fluctuations in λ to be most important.

Simulation frameworks for RL, such as MuJoco (Todorov et al., 2012), specify a control frequency, which determines the actual control frequency f^{RC}, i.e., the time between control actions as well as the waiting time $\Delta t_{(a_t, s_{t+1})}$ between applying an action a_t and collecting the next state s_{t+1}. These simulations assume no time spent on communication, action computation, or gradient updates, i.e., $\Delta t_{(a_t, s_{t+1})} = (f^{\mathrm{RC}})^{-1}$. These assumptions are unrealistic in the real world, and the waiting time can only use up some fraction of the control frequency. This poses an inherent dilemma, as a faster control frequency can only be achieved with a lower waiting time. Sufficient waiting times, however, are critical for allowing the RL agent to observe the consequences of its actions. To balance hardware limitations with the need for adequate waiting time, we selected a control frequency of 20 Hz and a waiting time of 0.033 s. In simulation, a higher control frequency of 50 Hz allowed for improved responsiveness to force changes. In addition, we defined a split actor and learner setup, similar to Luo et al. (2024), to ensure that gradient updates do not lower the control frequency. This also allows the environment to reset parallel to the gradient steps.

The second challenge is the measurement of noise in the robot state and the force-torque sensor. In the latter, we encountered drift and motion-dependent sensor noise. This high noise level, combined with the low control frequency, complicates the training on hardware. We applied a low-pass filter with a cut-off frequency of 35 Hz. This simplified the optimization landscape and enabled the agent to focus on aspects within control.

In our simulation study, we found one critical aspect specific to the CHEQ algorithm and the VIC task. In the initial training phase, the weighting factor fluctuates greatly between time steps, resulting in highly fluctuating IC gains. In simulation, this does not pose a problem. On real hardware, however, the gain fluctuations lead to a chattering end-effector motion. This doubly affects the training as it complicates the control and increases the motion-dependent force sensor noise. To mitigate this, we use an average of 10 time steps to compute our uncertainty measurement, leading to a smoother weighting.

Combining the abovementioned challenges, we find that data collected on our hardware setup is less reliable than in simulation. This can lead to unstable training. However, CHEQ provides a mechanism to mitigate this by slowing the curriculum. Thus, for our hardware experiments, we set the uncertainty limits to be more conservative. This reduces the agency of the RL agent in the beginning and leads to more stable, even safer, albeit slower, learning.

Additional Hardware Results. We show detailed hardware results in Fig. C.1. The return distribution of the early training phases shows that the return rises fast and surpasses the control prior (see Fig. C.1a). The complete progression of training failures can be found in Fig. C.1b. We further show the force, velocity and weight behavior of an evaluation run after more training steps (see Fig. C.1c). This underlines our findings that the agent learns to act with a higher weighting λ^{RL} but does not further improve the polishing behavior.

Controller	k_x	k_y	k_z	k_{ϕ_x}	k_{ϕ_y}	k_{ϕ_z}	ζ
Nominal (untuned)	500	160	50	500	500	500	1.0
Nominal (tuned)	500	107	68	500	500	500	0.9562
Nominal (hardware, untuned)	800	1085	900	150	80	25	1.0

Table A.6. Definition of impedance gains for the fixed gain nominal controllers.

partition	k_y [N m^{-1}]	k_z [N m^{-1}]	ζ
S1	98	59	~ 0.9445
S2	91	69	~ 0.8563
S3	108	71	~ 0.9452
S4	137	72	~ 1.0294
S5	101	65	~ 1.1946

Table A.7. Impedance profile for the *Nominal (tuned section-wise)* controller.

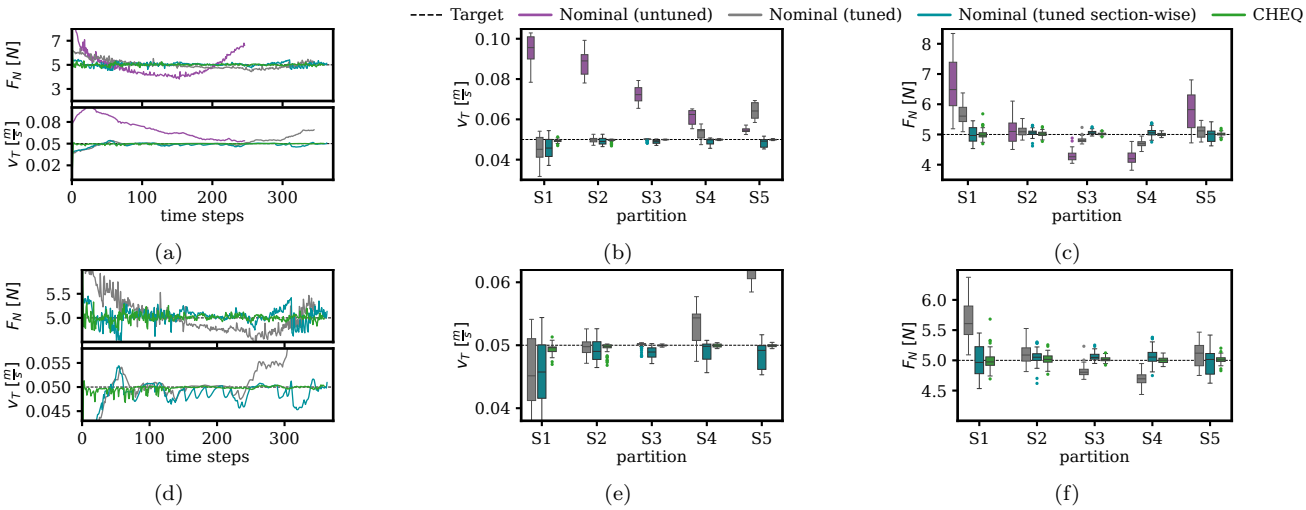
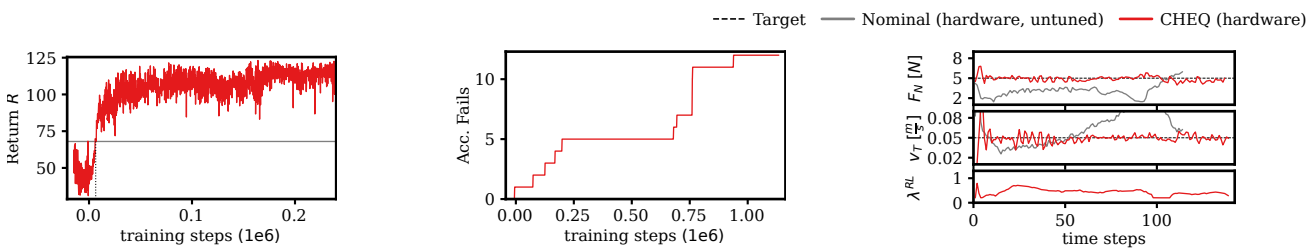


Fig. B.1. Comparison of the trained CHEQ agent with the nominal control baselines.



(a) Initial training return.

(b) Accumulated fails (log scale).

(c) Late evaluation: $\sim$ 930000

Fig. C.1. We show a close look at the initial training return in Fig. (a). Further, the accumulated training fails are shown in Fig. (b). Figure (c) shows the force and velocity behavior of an evaluation run after longer training ($\sim$ 930000 training steps).